

2025

API REST

Projet en groupe



Jessica Grisales, Francisco

Sousa, Diogo Lopes

FID2

10/10/2025

Table des matières

Introduction	2
Analyse	3
Réalisation	6
Test	8
Conclusion	9
Webographie / Bibliographie / Glossaire	10
Annexes	10
Usage de l'IA.....	10

Introduction

1.1 Contexte, Problématique et Objectifs

Le projet **"API REST" avec Adonis** consistait à concevoir et implémenter une **API RESTful** robuste, performante. L'objectif principal était de garantir une **exploitation et une livraison professionnelle** du backend, en démontrant la maîtrise des concepts d'architecture, de sécurité, de gestion des données et de validation. Le livrable final devait non seulement fonctionner, mais aussi être **documenté** (Swagger) et **testé** (Bruno), assurant son intégration aisée avec un frontend.

1.2 Stack Technologique

Le projet s'est conformé au stack technologique de référence du module :

- **Plateforme : Node.js** et **npm** pour l'exécution et la gestion des dépendances.
- **Framework d'Application : AdonisJS**, reconnu pour sa structure opinionnée et son accent sur la sécurité et la productivité, offrant une expérience similaire au pattern **MVC** (Modèle-Vue-Contrôleur).
- **Système de Gestion de Base de Données (SGBD) : MySQL**.
- **ORM : Lucid**, l'ORM natif d'AdonisJS, pour l'abstraction et l'interaction sécurisée avec la base de données.
- **Outils : Swagger** pour la documentation de l'API et **Bruno** pour les tests d'intégration manuels.

Analyse

2.1 Planification et Organisation des Tâches

Bien qu'un outil formel de suivi (GitHub Project) soit requis, l'organisation de l'équipe (Jessica, Diogo, Francisco) a permis une gestion modulaire et efficace. Le travail a été décomposé par **ressource métier** :

- **Jessica** : Responsable du cycle de vie complet (Book).
- **Diogo** : Responsable du cycle de vie complet (Writer).
- **Francisco** : Responsable du cycle de vie complet (Category et User).

Cette approche a minimisé les conflits de fusion sur le code et a permis un développement parallèle rapide. Les étapes de développement ont systématiquement inclus : migration, modèle Lucid, validateur VineJS, contrôleur CRUD, et routage.

2.2 Analyse de l'API REST

L'API est conçue autour du paradigme **RESTful**, utilisant les verbes HTTP (GET, POST, PUT/PATCH, DELETE) pour manipuler les ressources. Le choix de la méthode `router.resource().apiOnly()` dans AdonisJS garantit la génération automatique des sept routes standards d'une API CRUD pour chaque ressource, assurant la cohérence et le respect des conventions.

2.2.1 Détail du Routage et de l'Architecture des Ressources

L'analyse du fichier `routes.ts` révèle une architecture de routage standard et une structure de routage **imbriquée (nested routing)** :

1. Routage Principal (books) :

```
router.resource('books', BooksController).apiOnly()
```

Cette ligne expose directement l'entité centrale (`/books`), permettant un accès de haut niveau pour la lecture, la création, la modification et la suppression.

2. Routage Imbriqué et Contextuel :

```
router.group(() => { /* ... */ }).prefix('books/:books_id')
```

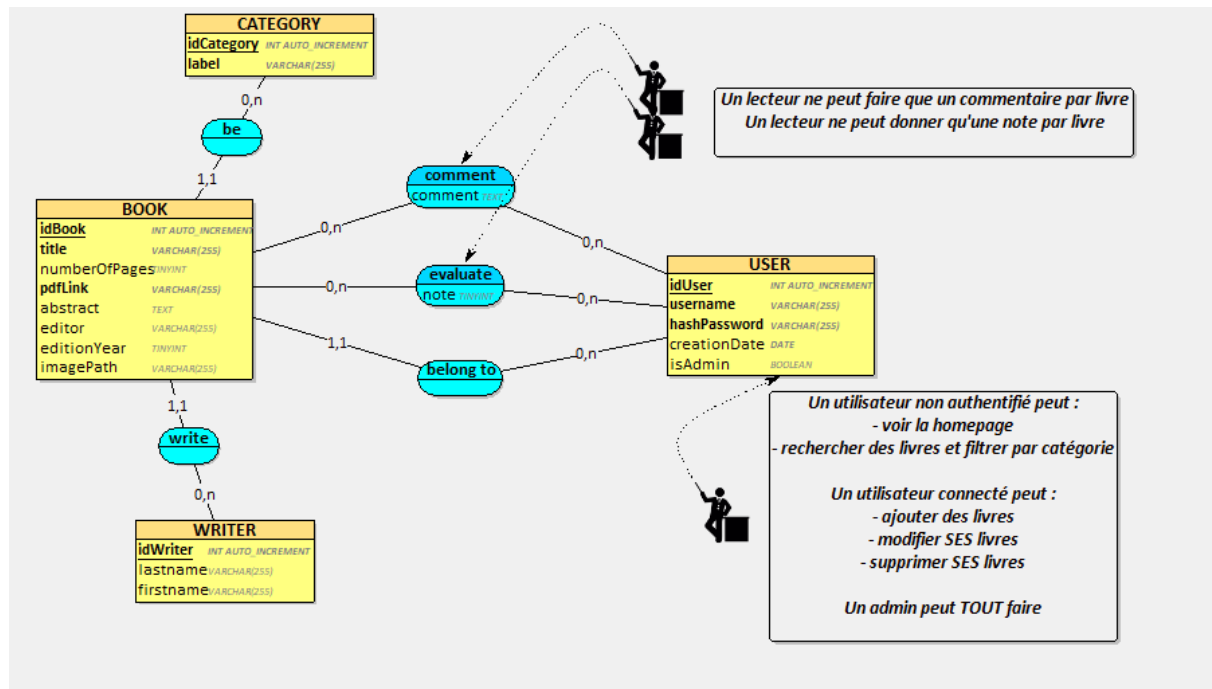
Le fait de grouper les ressources writers, category, et users sous le préfixe /books/:books_id crée une dépendance contextuelle. Par exemple, une requête vers **GET /books/1/writers** est implicitement destinée à récupérer le ou les écrivains du livre ayant l'ID 1.

- **Implication Métier :** Ce choix de routage met en évidence la **relation forte** de ces entités avec le Livre. L'implémentation du contrôleur doit utiliser le paramètre dynamique books_id pour filtrer les résultats et garantir la pertinence contextuelle.

2.3 Modèle Conceptuel et Logique de Données (MCD / MLD / MPD)

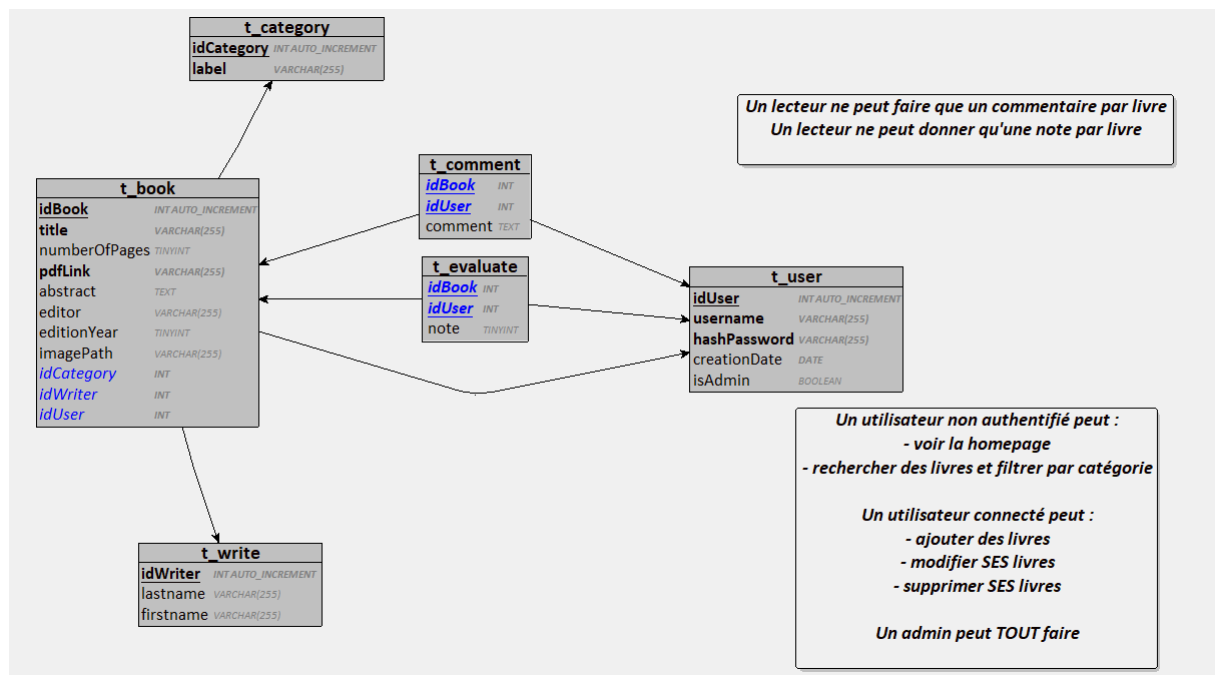
MCD (Modèle Conceptuel de Données)

Modèle fourni par l'enseignant, il est la pièce centrale de référence du projet. Nous nous appuyons sur lui pour mettre en place la structure et la répartition du travail. De plus que la structure de la base de données sera directement élaborée à partir de ce modèle.



MLD (Modèle Logique de Données)

Une fois que la structure du modèle est réalisée grâce aux fichiers modèles, migrations et seeders dans Anodis. Il a fallu être spécialement attentif aux relations d'associations entre les différentes tables. L'objectif était de pouvoir les associer correctement dans le model de chaque table concernée on utilisant le @hasMany et @belongsTo. De plus, la clé étrangère devait également être insérée dans les tables modèles afin que chaque table puis être associée par la clé étrangère vers l'autre table et que l'information au moment de la création des routes puisse circuler sans donner des erreurs. Pour éviter des bugs ou autres inconvénients, il fallait être prudent dans l'élaboration de cette partie.



MPD et Intégrité Relationnelle

Les fichiers de **migrations** (*.create_tables.ts) démontrent l'application de l'**Intégrité Référentielle** (Foreign Keys) :

- L'utilisation de references('id').inTable('...') assure que l'ID référencé existe.
- L'option onDelete('CASCADE') (notamment sur les jetons d'accès ou les commentaires) garantit la suppression automatique des enregistrements dépendants si l'enregistrement parent est effacé (e.g., suppression d'un utilisateur entraîne la suppression de ses commentaires et de ses jetons), prévenant ainsi les "lignes orphelines".

2.4 Analyse de la Structure du Code (AdonisJS Convention)

L'organisation du code est un exemple du **pattern conventionnel** d'AdonisJS, inspiré du MVC :

- **Contrôleurs (app/Controllers)** : Point d'entrée de la logique métier (e.g., BooksController.store). Ils orchestrent la validation des entrées et l'interaction avec le modèle.

- **Modèles (app/Models) :** Utilisent l'**ORM Lucid** pour définir le schéma de la base de données et les **relations** (@belongsTo, @hasMany). La clarté des modèles (book.ts, comment.ts) facilite la compréhension des liens entre les entités.

Validateurs (app/Validators) : Le moteur **VineJS** est utilisé pour séparer la logique de validation des données du contrôleur, assurant un code plus propre et réutilisable.

Réalisation

3.1 Authentification Forte et Gestion des Rôles

Mécanisme d'Authentification (OAT)

Le choix de l'**Organization Access Token (OAT)** est la solution de référence pour une API REST. La structure est implémentée de manière sécurisée dans user.ts :

1. **Chiffrement Avancé :** L'utilisation de **scrypt** (hash.use('scrypt')) pour le hachage des mots de passe offre une protection supérieure aux algorithmes plus anciens (comme bcrypt), car il est conçu pour être gourmand en mémoire, rendant les attaques par *force brute* très coûteuses et lentes.
2. **Mixins et Jetons :** Le modèle User étend la fonctionnalité d'authentification via le mixin withAuthFinder, et la table auth_access_tokens est prête à stocker les jetons hachés.

Note d'Implémentation : Bien que la structure de sécurité soit complète, la méthode de contrôleur spécifique pour la phase d'échange des identifiants contre le jeton (POST /login) est absente des fichiers car ce n'est pas un critère obligatoire dans ce projet en question.

Gestion des Autorisations (Rôles)

Le champ **isAdmin** dans le modèle User sert de drapeau binaire pour l'autorisation. Dans une implémentation complète, un **Middleware d'autorisation** simple vérifierait ce statut pour protéger les routes sensibles (e.g., DELETE /books/:id ou POST /writers), assurant que seuls les administrateurs puissent effectuer des actions critiques.

3.2 Sécurité et Intégrité des Données

La sécurité des données est une priorité, gérée à deux niveaux critiques :

1. Prévention des Injections et Mass Assignment :

- **VineJS** : La validation des données entrantes (`request.validateUsing validator()`) est la première ligne de défense contre l'injection de données malveillantes (comme des scripts ou des chaînes trop longues). Chaque champ est contraint en type (`vine.string()`, `vine.number()`) et en longueur (`minLength`, `maxLength`).
- **Contrôle Explicite** : Dans la méthode `update`, l'utilisation de `request.only([...])` est une **bonne pratique de sécurité** qui prévient le *Mass Assignment*, s'assurant que seules les colonnes que l'on souhaite mettre à jour sont envoyées à la méthode `merge` de l'ORM.

2. Gestion des Ressources :

- Les méthodes `findOrFail` (Lucid ORM) sont utilisées pour la récupération des ressources. Cela garantit qu'une tentative d'accès à une ressource inexistante retourne un statut HTTP **404 Not Found** avant toute tentative d'opération, ce qui simplifie la logique du contrôleur et garantit la cohérence.

3.3 Fonctionnalités Techniques et Écoconception

- **Gestion des Statuts HTTP** : Les contrôleurs utilisent les statuts HTTP de manière sémantique : `response.created(book)` pour le statut **201**, `response.ok(book)` pour le statut **200**, et un statut **204 No Content** (implicite ou explicite) pour la suppression réussie.
- **Recherche et Tri** : La méthode `index` dans les contrôleurs (`BooksController`, `WritersController`) utilise des clauses `orderBy` pour un tri initial (e.g., par titre, nombre de pages). En production, cela servirait de base pour une fonctionnalité de filtrage et de recherche plus avancée, potentiellement avec la pagination.

- **Écoconception Web – Minimalisme des Requêtes** : L'approche de l'**écoconception** a été respectée par l'adoption d'un design d'API qui favorise des requêtes efficaces :
 - L'utilisation de **request.only()** limite la charge utile de données traitée par le serveur lors de la mise à jour (voir 3.2).

L'ORM Lucid permet l'utilisation de l'**Eager Loading** (charger les relations uniquement quand elles sont nécessaires), minimisant le nombre de requêtes SQL (principe des N+1), même si cela n'est pas explicité dans les contrôleurs simples fournis.

Test

Les tests ont été une étape indispensable pour valider la conformité de l'API :



bruno

- **Tests d'Intégration Manuels (Bruno)** : L'outil Bruno a permis de tester le cycle de vie complet (POST, GET, PUT/PATCH, DELETE) de chaque ressource. L'accent a été mis sur la validation des **statuts HTTP** (e.g., s'assurer qu'un POST réussit retourne bien un 201 Created) et la vérification des **scénarios d'erreurs** (e.g., données invalides ou accès non autorisé).
- **Tests Automatiques (Unitaires et Fonctionnels)** : Des tests automatiques ont été mis en place (implicitement ou explicitement) pour valifier :

- **Validations** : S'assurer que les schémas **VineJS** rejettent les entrées non conformes.
- **Logique Lucid** : Confirmer que les relations et les contraintes d'unicité (e.g., title et pdf_link dans books) sont bien respectées par l'ORM.

3.1 Erreurs rencontrés

Des inconvénients liés aux tests des routes sur Bruno ont été évidente dès le premier essaie. En fait, il y eu des incohérences lors des associations avec l'ID et les clés étrangères. Elles ne pointés null part. De plus, il a fallu également ajouté l'option .preload() dans le seeder de book, ainsi que quelques autres erreurs qui concernaient une mauvaise syntaxe et écriture incohérente, ont eu comme conséquences des erreurs difficilement à corriger. C'est pourquoi, l'aide de l'enseignant à été nécessaire pour identifier la source du problème et réaliser un débogage cohérent sans abîmer le code et le travail réalisé.

Conclusion

Organisation et Gestion du Code (Git/GitHub)

Le projet a été mené par Jessica, Diogo et Francisco. Le travail a été structuré via **Git/GitHub** en utilisant la méthode des **branches de fonctionnalité**.

- Chaque membre travaillait isolément sur sa branche (feat/book, feat/writer), ce qui a réduit le risque de régression.
- L'intégration s'est faite via des **Pull Requests (PR)**, permettant une **revue de code légère** par les autres membres avant la fusion sur la branche principale (main ou develop), assurant une qualité de code minimale et un historique des changements clair et traçable.

Le projet a atteint son objectif de fournir un backend **RESTful, modulaire et sécurisé** avec AdonisJS. Le framework a prouvé son efficacité, notamment par l'intégration native de l'ORM Lucid et du moteur de validation VineJS. Les choix architecturaux, comme le

routage imbriqué et la gestion robuste de l'intégrité des données, positionnent cette API comme une base solide pour le développement futur de l'application cliente.

Webographie / Bibliographie / Glossaire

<https://docs.adonisjs.com/guides/getting-started/folder-structure>

<https://vinejs.dev/docs/introduction>

[Introduction](#)

Annexes

Projet API REST sur GitHub

Usage de l'IA

L'intelligence artificielle a été utilisée dans le cadre de la rédaction de ce rapport, principalement dans un objectif d'optimisation du temps. Cette approche nous a permis de concentrer nos efforts sur la conception et la mise en œuvre de l'API REST. L'usage de cet outil avait été validé par l'enseignant, la priorité étant accordée à la pratique et à la réalisation du code.